```
    })
    function start() {
      observer.observe(target, { childList: true, subtree: true })
    }
    start()
  }

  sortTargets(): void {
    if (this.targetsAlreadySorted()) {
      return
    }
    this.sortElementTargets
      .sort((a: HTMLElement, b: HTMLElement) => {
        return this.sortValue(a) - this.sortValue(b)
      })
      .forEach((element: HTMLElement) => this.element.append(element))
  }

  targetsAlreadySorted(): boolean {
    let [first, ...rest] = this.sortElementTargets
    for (const next of rest) {
      if (this.sortValue(first) > this.sortValue(next)) {
        return false
      }
      first = next
    }
    return true
  }

  sortValue(element: HTMLElement): number {
    return parseInt(element.dataset.sortValue, 10)
  }
}
```

The first thing to notice in our SortController is the initialize method, which is called immediately when the controller is loaded, as opposed to create, which is called when an instance of the controller is detected in the DOM.

This initialize method is a little tangled, but it does three things:

- Creates a new MutationObserver object, and passes a callback function as an argument to that constructor